RUSTIKON
Powered by SoftwareMill

exe1n

Running 0 times per second for maximum speed

Wojciech Brożek

RUSTIKON
Powered by SoftwareMill

exe1n

github.com/Niedzwiedzw
wojciech.brozek@niedzwiedz.it

```
/// `https://exein.io`
trait Exein: Rust + Security {}
```

60FPS

60FPS (~16ms per frame)

Solution: doing less at runtime!

Solution: doing less at runtime!

(some nightly features)

Solution: doing less at runtime!

(some nightly features)

(some unsafe code)

Joining *&'static str*-ings at runtime.

Joining *&'static str*-ings at runtime.

`concat!("a", "b")`: only accepts literals (not very flexible)

```rust
/// build a &'static str inside `const {}`
///
/// implementation:
/// `github.com/Niedzwiedzw/`
struct ConstStr {
    // ..
}
```

```rust
impl ConstStr {
    /// Initialize in `const {}`
    pub const fn new() -> Self {}
    /// push strings in `const {}`
    pub const fn push_str(self, string: &'static str) -> Self {}
    /// safely cast to `&'static str` in `const {}`
    pub const fn as_str(&'static self) -> &'static str {}
}
```

HTML builder... in Rust’s type system!

RUSTIKON
Powered by SoftwareMill

exe1n

```rust
// The dream - this Rust code...
el("main")
  .attribute("id", "rustikon")
  .child(
    el("form")
      .child(
        el("input")
          .attribute("value", "hello Rustikon 2026!")
      )
  )
```

```html
<!-- ...produces this html - at compile time -->
<main id="rustikon">
  <form>
    <input value="hello Rustikon 2026"></input>
  </form>
</main>
```

```
// ./src/lib.rs
// + #![feature(adt_const_params)]
// + #![feature(unsized_const_params)]
```

```rust
// Notice: no methods...
pub trait IsAttribute {
    const ATTRIBUTE: ConstStr;
}


// ...just slots to store &'static str-s
pub trait IsChild {
    const CHILD: ConstStr;
}
```

```rust
pub trait IsChildren {
    const CHILDREN: ConstStr;
}
pub trait IsAttributes {
    const ATTRIBUTES: ConstStr;
}
```

```rust
/// For example: `style="background: black"`
pub struct Attribute<
    const K: &'static str,
    const V: &'static str,
>;
```

```rust
impl<
  const K: &'static str,
  const V: &'static str,
> IsAttribute for Attribute<K, V> {
    const ATTRIBUTE: ConstStr = ConstStr::new()
        .push_str(K)
        .push_str(r#"=""#)
        .push_str(V)
        .push_str(r#"""#);
}
```

```rust
pub struct Empty;

impl IsChildren for Empty {
    const CHILDREN: ConstStr = ConstStr::new();
}

impl IsAttributes for Empty {
    const ATTRIBUTES: ConstStr = ConstStr::new();
}
```

```rust
pub struct PushAttribute<
    T: IsAttributes,
    A: IsAttribute,
>(PhantomData<(T, A)>);

impl<
  T: IsAttributes,
  A: IsAttribute,
> IsAttributes for PushAttribute<T, A> {
    const ATTRIBUTES: ConstStr = ConstStr::new()
        .push_str(T::ATTRIBUTES.as_str())
        .push_str(" ")
        .push_str(A::ATTRIBUTE.as_str());
}
```

```rust
pub struct PushChild<T, C>(PhantomData<T>, PhantomData<C>);

impl<T: IsChildren, C: IsChild> IsChildren for PushChild<T, C> {
    const CHILDREN: ConstStr = ConstStr::new()
        .push_str(T::CHILDREN.as_str())
        .push_str(C::CHILD.as_str());
}
```

```rust
impl<const TAG: &'static str, A, C> DomTree<TAG, A, C>
where
    A: IsAttributes + 'static,
    C: IsChildren + 'static,
{
    pub const fn child<Child: IsChild + 'static>(
        self,
        _child: Finished<Child>,
    ) -> DomTree<T, Attributes, PushChild<C, Child>> {
        DomTree::new()
    }
```

LIVE DEMO!